Análise de Algoritmos e Complexidade

Universidade Estadual do Sudoeste da Bahia

2026.1

Analogia

- **Cenário:** Transferir um arquivo para o outro lado do país.
- **Via Internet:** O tempo de transferência cresce com o tamanho do arquivo. Arquivos gigantes demoram meses. Chamamos a isso de $O(N)$.
- **Via Avião:** O tempo de voo é o mesmo para 1 byte ou 1 Petabyte. Chamamos a isso de $O(1)$ (Tempo Constante).

A **Notação Big O** mede a taxa de crescimento do tempo à medida que a entrada aumenta, e não o tempo exato em segundos.

Notações

Na teoria rigorosa da computação:

- **Big O (O):** Limite Superior.
- **Big Omega (Ω):** Limite Inferior.
- **Big Theta (Θ):** Limite Restrito Exato, tanto O quanto Ω.

Nas entrevistas das Big Techs e na Programação Competitiva, quando pedem "Big O", estão quase sempre à procura do "Big Theta". Eles querem o tempo **exato**.

Melhor, Pior e Esperado

Exemplo Clássico: **Quick Sort**

- **Melhor Caso:** Os dados já ajudam. $O(N)$
- **Caso Esperado:** Ocorre na maioria das vezes. $O(N \log N)$
- **Pior Caso:** Escolha repetida do maior pivô. Degrada para $O(N^2)$

O juiz online constrói os casos de teste propositadamente para engatilhar o seu **Pior Caso**. Programe sempre pensando nele.

Complexidade de Espaço

Memória RAM também tem limites (MLE - Memory Limit Exceeded).

- Um array de N elementos exige $O(N)$ de espaço.
- Uma matriz $N \times N$ exige $O(N^2)$ de espaço.

Chamadas Recursivas também são levadas em consideração

Uma recursão com profundidade N que não aloca variáveis consome $O(N)$ na *Call Stack*.

Regras de Simplificação Matemática

1. **Abandone as Constantes:** $O(2N)$ torna-se simplesmente $O(N)$. Descartamos o coeficiente porque a taxa infinita importa mais.
2. **Abandone Termos Não Dominantes:** $O(N^2 + N)$ converte-se em $O(N^2)$.
3. **Soma vs. Multiplicação:**
   - Fazer A, **depois** fazer B $= O(A + B)$
   - Para **cada** A, fazer B (loops aninhados) $= O(A \times B)$

Dois Casos Especiais

- **Tempo $O(\log N)$:** Acontece quando o problema é cortado pela metade a cada passo (Ex: Busca Binária).
- **Árvore de Recursão:** Para a algoritmos recursivos, a regra é:

$$O(\text{ramificações}^{\text{profundidade}})$$

Calcular Fibonacci chamando $f(n-1) + f(n-2)$ tem 2 ramificações e profundidade N. O código terá complexidade de $O(2^N)$.

Prática

Regras:

Vou projetar um código C++.
Vocês têm **30 segundos**.
Anotem a complexidade (Big O) no caderno.

Exemplo 1

```
void printUnorderedPairs(int[] array) {
    for (int i = 0; i < array.length; i++) {
        for (int j = i + 1; j < array.length; j++) {
            cout << array[i] << ", " << array[j] << "\n";
        }
    }
}
```

Qual é a complexidade?

1. $O(N)$
2. $O(N^2)$
3. $O(N \log N)$

Resposta: Exemplo 1

Opção 2: $O(N^2)$

O código faz aproximadamente $\frac{N^2}{2} - \frac{N}{2}$ iterações. Ao aplicarmos as regras, cortamos as constantes e os termos não dominantes e sobra o loop quadrático.

Exemplo 2

```
void printPairs(int[] arrayA, int[] arrayB) {
    for (int i = 0; i < arrayA.length; i++) {
        for (int j = 0; j < arrayB.length; j++) {
            cout << arrayA[i] << ", " << arrayB[j] << "\n";
        }
    }
}
```

Qual é a complexidade?

1. $O(N^2)$
2. $O(A \times B)$
3. $O(A + B)$

Resposta: Exemplo 2

Opção 2: $O(A \times B)$

É muito comum chamar tudo de N. `arrayA` e `arrayB` são entradas diferentes. Nunca assuma que os tamanhos são iguais num problema a não ser que o enunciado o garanta explicitamente.

Exemplo 3

```
void reverse(int[] array) {
    for (int i = 0; i < array.length / 2; i++) {
        int other = array.length - i - 1;
        swap(array[i], array[other]);
    }
}
```

Inverte o array iterando apenas até meio. Complexidade:

1. $O(N)$
2. $O(N/2)$
3. $O(\log N)$

Resposta: Exemplo 3

Opção 1: $O(N)$

Embora itere $N/2$ vezes, descartamos a constante divisora. A taxa de aumento do tempo de execução continua crescendo de forma **linear** relativamente ao tamanho do array.

Exemplo 4

```
int f(int n) {
    if (n <= 1) return 1;
    return f(n - 1) + f(n - 1);
}
```

Qual é a complexidade desta recursão?

1. $O(N)$
2. $O(N^2)$
3. $O(2^N)$

Resposta: Exemplo 4

Opção 3: $O(2^N)$

A função ramifica 2 vezes a cada passo (*branches*) e desce N vezes (*depth*). Pela fórmula das árvores de recursão, ficamos com $O(2^N)$.

Regra para a Maratona

Como saber qual algoritmo usar antes de codificar?

$$10^8 \text{ operações por segundo}$$

Se o limite de tempo do Juiz é 1 segundo, a sua notação Big O, multiplicada pelo **tamanho máximo da entrada (N)**, não pode ultrapassar a marca de $\sim$ 100.000.000.

Tabela de complexidades aceitáveis

Faça engenharia reversa do N para descobrir a complexidade alvo:

Restrição ($N \leq \ldots$)	**Complexidade Alvo**	**Algoritmo Típico**
$N \leq 10$	$O(N!)$ ou $O(N^6)$	Força bruta (Permutações)
$N \leq 20$	$O(2^N)$	Força bruta recursiva / Bitmasks
$N \leq 100$	$O(N^3)$	Loops triplos (Floyd-Warshall)
$N \leq 1.000$	$O(N^2)$	Loops duplos (Bubble Sort, DP)
$N \leq 10^5$	$O(N \log N)$	Merge Sort / Segment Trees
$N \leq 10^6$	$O(N)$ ou $O(\log N)$	Iteração Simples / Busca Binária

Prática Intercalada: Adivinhe o Big O

Cenário 1 (Apple Division): Dividir maçãs em 2 grupos para minimizar a diferença de peso.
Restrição: $N \leq 20$. Peso $\leq 10^9$.

Cenário 2 (Shortest Routes): Mapa de rotas. Responder à menor distância de A a B.
Restrição: $N \leq 500$ cruzamentos.

Respostas: Cenários 1 e 2

Cenário 1 (Apple Division): Dividir maçãs em 2 grupos para minimizar a diferença de peso.
Restrição: $N \leq 20$. Peso $\leq 10^9$.
Resposta Esperada: $O(2^N)$ **Cenário 2 (Shortest Routes):** Mapa de rotas.

Responder à menor distância de A a B. **Restrição:** $N \leq 500$ cruzamentos.
Resposta Esperada: $O(N^3)$

Prática Intercalada: Adivinhe o Big O

Cenário 3 (Sum of Two Values): Achar dois itens no array cuja soma seja X. **Restrição:** $N \leq 2 \cdot 10^5$.

Cenário 4 (Creating Strings): Imprimir todas as combinações de letras embaralhadas. **Restrição:** $N \leq 8$.

Respostas: Cenários 3 e 4

Cenário 3 (Sum of Two Values): Achar dois itens no array cuja soma seja X.
Restrição: $N \leq 2 \cdot 10^5$.
Resposta Esperada: $O(N \log N)$ ou melhor

Cenário 4 (Creating Strings): Imprimir todas as combinações de letras embaralhadas.
Restrição: $N \leq 8$.
Resposta Esperada: $O(N!)$

Prática Intercalada: Adivinhe o Big O

Cenário 5 (Weird Algorithm): Se o número for par, $N/2$. Se ímpar, $N \times 3 + 1$. Quantos passos até chegar a 1?
Restrição: $N \leq 10^{14}$.

Pergunta: Pela tabela, precisaríamos de $O(1)$ ou $O(\log N)$. Será?

Resposta: Cenário 5

Cenário 5 (Weird Algorithm): Se o número for par, $N/2$. Se ímpar, $N \times 3 + 1$. Quantos passos até chegar a 1?
Restrição: $N \leq 10^{14}$.

Pergunta: Pela tabela, precisaríamos de $O(1)$ ou $O(\log N)$. Será?

Resposta: Não. Embora o N seja gigante, o problema converge rapidamente para 1. A simulação pura é aceita.

Fechamento

Peguem papel e caneta ou abram o bloco de notas.

1. Anotem a tabela baseando-se apenas na memória.
2. Relacionem o tamanho do N com a Complexidade de tempo correspondente.

Sempre analise a restrição do N antes de começar a programar.